Full Stack Development Summary

INHERITANCE

Advantages of Inheritance

Code Reusability

Avoid rewriting common logic.

Easy Maintenance

Updates in the superclass are reflected in subclasses.

Extensibility

Allows adding new features without changing old code.

Method Overriding

Enables polymorphism (same method, different behavior).

Organized Code

Promotes modular and structured programs.

Types of Inheritance in Java

- Single Inheritance.
- Multilevel Inheritance.
- Hierarchical Inheritance.
- Multiple Inheritance.
- Hybrid Inheritance.

Single Inheritance

A → B

One class inherits from another

Example Program

Multilevel Inheritance

A → B → C

Class inherits from a subclass

Example Program

2 Multilevel Inheritance

Grandparent → Parent → Child

Definition:

In multilevel inheritance, a class is derived from another subclass (i.e., inheritance in steps).

```
class Grandparent {
    void home() {
        System.out.println("Grandparent's house");
    }
}

class Parent extends Grandparent {
    void car() {
        System.out.println("Parent's car");
    }
}

class Child extends Parent {
    void bike() {
        System.out.println("Child's bike");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.home();
        c.car();
        c.bike();
    }
}
```

```
Grandparent's house
Parent's car
Child's bike
```

Hierarchical Inheritance

A → B, A → C

Multiple classes inherit from one superclass

Example Program

3 Hierarchical Inheritance

Definition:
When **multiple classes inherit** from a **single parent class.**

```
    Animal
    /    \
Dog      Cat
```

```java
class Animal {
    void eat() {
        System.out.println("All animals eat.");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks.");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows.");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();

        d.eat();
        d.bark();
        c.eat();
        c.meow();
    }
}
```

```
All animals eat.
Dog barks.
All animals eat.
Cat meows.
```

Multiple Inheritance

A, B → C

A class inherits from multiple classes (**Not allowed with classes**, allowed with interfaces)

Example Program

Example

```java
interface A {
    void show();
}

interface B {
    void display();
}

class C implements A, B {
    public void show() {
        System.out.println("Show from A");
    }

    public void display() {
        System.out.println("Display from B");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.show();
        obj.display();
    }
}
```

```
Show from A
Display from B
```

Hybrid Inheritance

Combination of above types.

Combination of above types (**Not directly supported**, achieved using interfaces)

Example Program